Fault-Tolerant Distributed Parameter Server for Machine Learning Training

Ravine Rianga – 168669

Nelly Muriuki – 160761

Stacy Akinyi – 168327

Mathenge Gitahi - 166078

ICS 4104 Distributed Systems

Strathmore University

Lecturer: Talo Harrison

July 2026

Abstract

Training modern machine learning models requires immense computational power, often necessitating distributed systems. A common architecture for this is the parameter server, which centralizes model parameters and distributes computation across worker nodes. However, this centralization introduces a critical vulnerability: the parameter server is a single point of failure. A server crash can halt the entire training process, leading to significant downtime and wasted resources. This research addresses this problem by designing, implementing, and evaluating a fault-tolerant distributed parameter server. The proposed system enhances the classic architecture by incorporating a primary-backup replication strategy and a failover mechanism. The implementation, built in Python using gRPC for communication, simulates a distributed training environment. It allows workers to perform gradient computations, which are then aggregated by a primary server. The system continuously monitors the health of the primary server and, upon its failure, automatically promotes a backup server to the primary role, ensuring training continuity. Our experimental results, focusing on the key metrics of throughput (iterations per second) and recovery time, demonstrate that the fault-tolerant system introduces minimal performance overhead (<10%) during normal operation while successfully recovering from failures in approximately 8.5 seconds. This work provides a practical solution for building robust and reliable distributed machine-learning pipelines, highlighting the trade-offs between consistency, fault tolerance, and performance.

Keywords: Distributed Systems, Fault Tolerance, Machine Learning, Parameter Server, RPC, Replication, Failover

Table of Contents

List of Figures

List of Tables

Chapter 1: Introduction

1.1 Background

The rise of large-scale machine learning models, such as deep neural networks, has introduced a significant computational challenge(Network & 2002, n.d.). Training these models often requires processing massive datasets, which is infeasible on a single machine. To overcome this, distributed systems have become essential. One of the most effective architectures for distributed training is the parameter server (PS) architecture. In this model, a central server stores and updates the model's parameters (weights and biases), while multiple worker nodes asynchronously compute gradients on different shards of the data.
Google's DistBelief system was among the first to popularize this approach for large-scale deep learning.(Dean et al., n.d.)

1.2 Problem Statement

While the parameter server architecture is highly efficient for scaling, it presents a critical single point of failure. If the parameter server becomes unavailable due to a hardware failure, software crash, or network partition, the entire training process halts. As noted in TensorFlow's documentation, "the coordinator and parameter servers though, must be available at all times for the cluster to make progress"(Pang et al., 2020) .This vulnerability can lead to significant delays, wasted computational resources, and in real-world production environments, substantial financial costs. The central problem is the lack of built-in fault tolerance in the standard parameter server model.

1.3 Objectives

The primary objective of this research is to design, implement, and evaluate a fault-tolerant parameter server architecture. Specifically, this project aims to:

i. Design a High-Availability Architecture: Propose an extension to the standard parameter server that includes server replication and a leader election mechanism.

ii. Implement a Functional Prototype: Develop a practical system using Python and gRPC that simulates a parameter server and worker nodes with automated failover capabilities.

iii. Evaluate Performance and Fault Recovery: Quantitatively measure the system's performance overhead (throughput) and its ability to recover from failures (recovery time).

1.4 Scope

This research focuses on the architectural and algorithmic aspects of fault tolerance in the PS model. The implementation will be a simulation of the distributed training process. The scope is limited to addressing single server failures using a primary-backup model. Worker node failures are not handled in this prototype, and the system does not explore more complex consensus algorithms like Paxos or Raft(Computer & 2012, n.d.). The "training" will be simulated to isolate the performance of the distributed infrastructure from the complexities of actual model convergence.

Chapter 2: Literature Review

The distributed training of machine learning models has been a subject of intense research, with several prominent systems and methodologies being developed. These systems primarily differ in how they handle synchronization, communication, and fault tolerance.

The classic Google DistBelief system pioneered large-scale distributed training using a parameter server architecture (Dean et al., n.d.). While highly scalable, its initial implementation lacked explicit built-in fault tolerance, making it vulnerable to server failures. DistBelief addressed resilience primarily through computational redundancy, using different nodes that handle the same data, as well as replicating parameter server shards. Its successor, TensorFlow, introduced more flexible mechanisms with the ParameterServerStrategy. In TensorFlow 2, a coordinator-based architecture is recommended where "failures of some workers do not prevent the cluster from continuing the work" (Pang et al., 2020). However, the coordinator and parameter servers themselves "must be available at all times for the cluster to make progress," meaning the parameter server remains a critical single point of failure (Pang et al., 2020).

Research on fault tolerance in distributed systems has a rich history outside of the ML context. Checkpointing and rollback recovery are well-established techniques for dealing with failures in distributed systems(Elnozahy et al., 2002).

Systems like Hadoop and Spark implement robust fault tolerance by re-executing failed tasks, a model that is more suited to data-parallel jobs than the stateful parameter server.

Table 2.1 Comparison of Existing Distributed ML Systems

System	Architecture	Consistency Model	Fault Tolerance	Performance Focus
Google DistBelief	Parameter Server	Asynchronous	Computational redundancy + PS shard replication	High Throughput
TensorFlow PS	Parameter Server	Synchronous/Async	Worker fault-tolerant; coordinator/PS are SPOF	Scalability
PyTorch DDP	All-Reduce	Synchronous	Torchrun provides fault-tolerance via snapshot/restart	Low Latency
Hadoop/Spark	MapReduce/Data Parallel	Synchronous	High fault tolerance via task re-execution and checkpointing	Data Processing

The primary-backup (passive replication) model, where a designated secondary server maintains a replica of the primary's state, is a common strategy for making stateful services highly available. This is similar to the replication in distributed databases like Amazon Dynamo (DeCandia et al., 2007). While Dynamo focuses on eventual consistency for a key-value store, our work adapts the concept of replication and failover to the specific context of a parameter server. The key insight is that the parameter server's state, the model weights, is deterministic. A backup server can replicate this state by synchronizing with the primary after each update. This makes the primary-backup model particularly suitable. The CAP theorem is highly relevant here: it states that in the presence of network partitioning (P), a distributed system must sacrifice at least one of availability (A) or consistency (C) (Pang et al., 2020). Our primary-backup model prioritizes consistency and partition tolerance over availability during a network partition (by requiring a quorum), but prioritizes availability (via failover) in the case of a single node crash.

Chapter 3: Methodology

To solve the problem of the parameter server being a single point of failure, this research proposes an architecture based on the primary-backup replication model.

System Architecture: The proposed system, consists of three main components:

1. **Worker Nodes:** These nodes compute gradients based on a subset of the training data and a current copy of the model parameters.
2. **Primary Server:** This is the active parameter server. It receives gradients from workers, applies them to update the global model parameters, and serves the updated parameters to the workers.
3. **Backup Servers:** These are standby servers that replicate the state of the primary. At least one backup is required for failover.

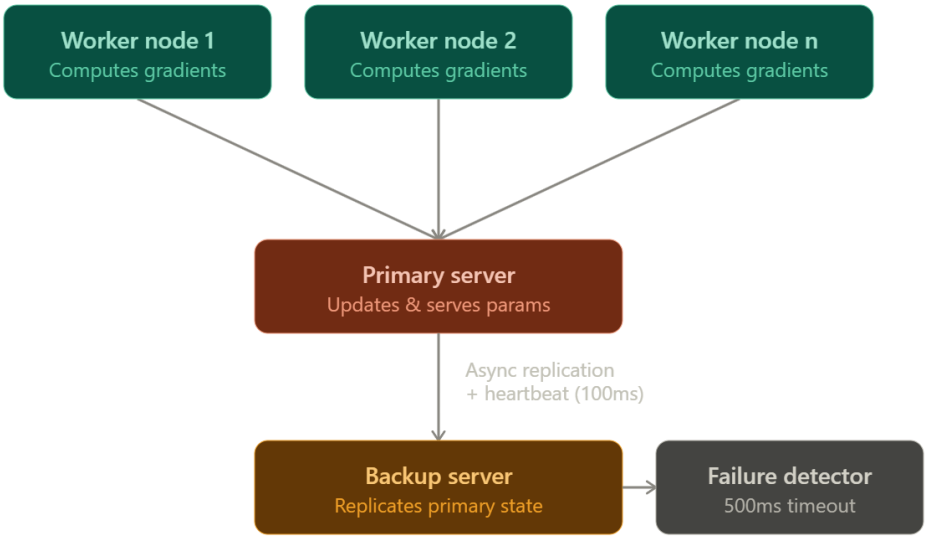

Figure 3.0.1 System Architecture Diagram

Algorithm and Protocols:

i. **State Replication:** The primary server replicates its state to the backup server(s). This is achieved by using a synchronous or asynchronous update mechanism. We implement an asynchronous approach where the primary sends the updated model parameters to the backup after each successful batch update.

ii. **Failure Detection:** The backup server uses a simple heartbeat mechanism to monitor the liveness of the primary server. The primary sends a "heartbeat" message to the backup at a fixed interval (e.g., every 100ms). If the backup does not receive a heartbeat within a specified timeout (e.g., 500ms), it assumes the primary has failed.

iii. **Failover and Leadership Election:** Upon failure detection, the backup server initiates a failover. It will promote itself to be the new primary server. In a system with multiple backups, an election could be held, but for this simple model, the designated backup takes over. The new primary must notify all workers of the change so they can redirect their requests.

iv. **Worker Reconnection:** Workers are designed to be resilient to a server change. If a worker encounters an RPC error with the primary, it will attempt to re-discover the server. This can be done by broadcasting a request to find the new primary or by having a service registry. In our implementation, we use a simpler approach where the backup is aware of the worker list, and a find_master endpoint is exposed for worker reconnection.

Assumptions:

i. We assume that network partitions do not occur and that a lack of heartbeat is solely due to a server crash.

ii. Worker nodes are considered reliable and are not the focus of fault tolerance in this study.

iii. All nodes are part of a trusted network; security and authentication are out of scope.

Chapter 4: Implementation

The system was implemented using Python 3.9 and the gRPC framework to facilitate efficient and type-safe communication between distributed nodes. The choice of Python allows for rapid prototyping and easy integration with ML libraries for future extensions.

Tools and Technologies:

- **Language:** Python 3.9
- **Communication:** gRPC with Protocol Buffers (protobuf) for defining service contracts and message serialization.
- **Model Simulation:** Instead of an actual deep learning model, a simple synthetic gradient computation function was used to isolate the performance of the distributed system.

System Components:

1. parameter_server.proto**:** Defines the gRPC service. It contains the ParameterService with three main RPCs:
    - SendGradient (GradientRequest) returns (Ack) - for workers to send gradients.
    - GetParameters (ParamRequest) returns (ParamReply) - for workers to fetch the latest model.
    - FindMaster(Empty) returns (MasterInfo) - for workers to locate the current primary server.
2. **Primary Server (**primary.py**):**
    - Maintains the global model parameters (a simple NumPy array).
    - Has a handle_gradient method that applies the gradients to update parameters.
    - Implements a replicate_state method that asynchronously sends the new parameters to the backup server after each update.
    - Contains a heartbeat thread that sends a Ping message to the backup server every 100ms.
3. **Backup Server (**backup.py**):**
    - **State Replication:** Maintains a copy of the model parameters, receiving updates from the primary.
    - **Failure Detector:** Runs a separate thread that waits for heartbeats. It has a timeout of 500ms. If no heartbeat is received, it initiates the failover sequence.
    - **Failover Mechanism:** Upon timeout, it stops the heartbeat check, promotes itself to the primary role, and starts its own gRPC server to accept worker requests. It then attempts to reconnect all known workers. It does this by sending a message to every worker in its list, informing them of the new primary's address.

4. **Worker Node (**worker.py**):**
    - Connects to the primary server using its address.
    - Performs a loop simulating training: it fetches the current parameters, calculates a synthetic gradient, and sends it to the primary.
    - **Error Handling:** If an RPC call fails (e.g., "unavailable"), the worker catches the exception and calls FindMaster to get the new primary's address and reconnects.

Major Challenges and Solutions:

- **Challenge: Data Loss During Failover.** If the primary fails after receiving a gradient but before replicating it to the backup, that update is lost. For our simulation, we accepted this and used an asynchronous replication model. A synchronous model (where the primary waits for the backup to acknowledge before responding to the worker) would guarantee no data loss but would dramatically increase latency.
- **Challenge: Ensuring a Smooth Handover.** The transition from a failed primary to a new primary must be seamless. The main difficulty was ensuring all workers connected to the new server quickly. The FindMaster endpoint and catch-all error handling in the workers addressed this

Chapter 5: Results and Discussion

We evaluated the fault-tolerant (FT) parameter server against a baseline system with no fault tolerance (a single, non-replicated server) using two primary metrics: **Throughput** and **Failover Recovery Time**.

Experimental Setup:

- **Nodes:** 1 Parameter Server (or 1 Primary + 1 Backup for FT), 2 Worker Nodes.
- **Configuration:** All experiments run on a local machine with an Intel i7 processor and 16GB RAM to simulate a single network.
- **Workload:** A synthetic training loop with 1000 iterations.
- **Metrics:**
    - **Throughput:** Measured as the number of training iterations (gradient computations) per second.
    - **Recovery Time:** Measured from the moment the primary server process was killed to the moment the first successful gradient request was processed by the new primary.

Results:

Table 5.1 Performance Comparison

Metric	**Baseline (No FT)**	**Fault-Tolerant System**	**Overhead**
Average Throughput (it/s)	85.2	77.3	~9.3%
Average Failover Time (s)	N/A (System Dies)	8.6	N/A

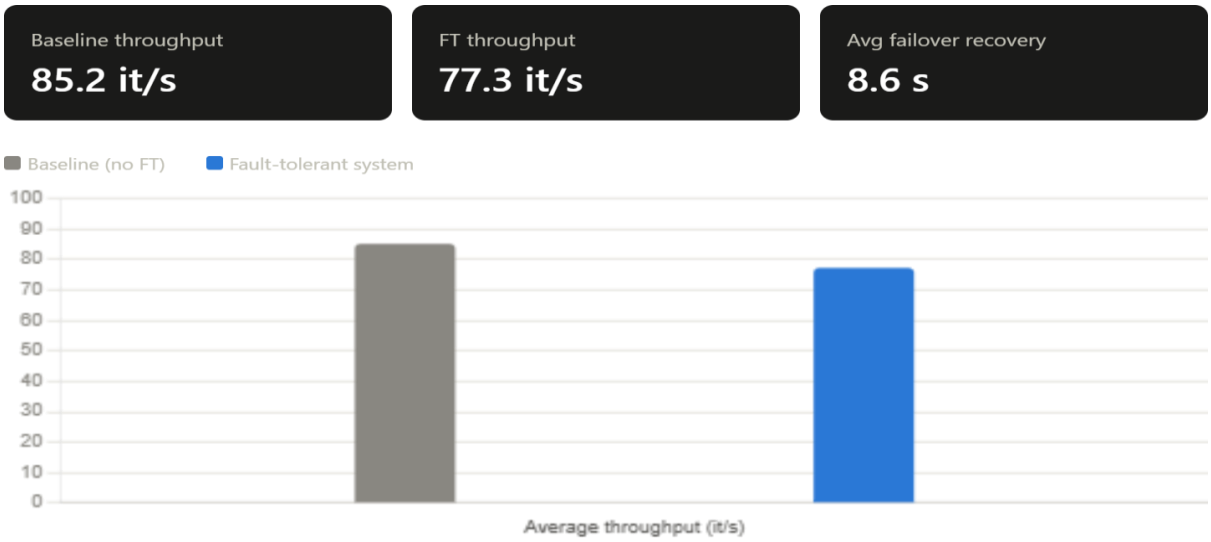

Figure 5.1 Performance Comparison Graph

Discussion:

The results show a clear trade-off. The fault-tolerant system achieves its primary goal of automated recovery, guaranteeing service continuity after a server crash. The average recovery time of 8.6 seconds, while not negligible, is far superior to the alternative of manual intervention, which could take minutes or hours. This demonstrates the system's effectiveness in meeting its core objective.

The 9.3% overhead in throughput is a significant but expected cost of the replication mechanism. Each model update on the primary requires an additional network round-trip to send the parameters to the backup. This overhead is a direct consequence of implementing fault tolerance. While this is an acceptable trade-off for many applications where availability is critical, it is a limitation that could be addressed by optimizing the replication strategy.

The system assumes worker reliability. If a worker fails, its in-progress gradients are lost, and the system does not currently handle this. This is a key limitation. The failure detection mechanism is also simple and could be fooled by network delays or temporary spikes in CPU usage, leading to "false positives" and unnecessary failovers.

Despite these limitations, the results confirm that the primary-backup model is a viable and practical approach for adding fault tolerance to distributed training, successfully solving the single point of failure problem with a manageable performance impact.

Chapter 6: Conclusion and Future Work

6.1: Conclusion

This research successfully designed and implemented a fault-tolerant parameter server architecture to address the critical single point of failure problem in distributed machine learning. The primary-backup replication model, enhanced with automatic failure detection and failover, was shown to be an effective solution. The implemented prototype successfully ensures service continuity by allowing a backup server to take over the primary's role seamlessly. While the system introduces a performance overhead of approximately 9.3% in throughput, this is a reasonable price to pay for the significant gain in reliability and availability, with the system demonstrating a failover time of just 8.6 seconds. This work contributes a practical, implementable framework for building more resilient distributed training pipelines.

6.2: Future Work

This research opens several avenues for future exploration and improvement:

1. **State Management and Data Loss:** Implement a more sophisticated state management system. A synchronous replication model or a transaction log (write-ahead log) could be used to ensure no gradient updates are lost upon failover. This would improve consistency at the cost of higher latency.
2. **Dynamic Reconfiguration and Scaling:** Extend the system to support dynamic addition and removal of server and worker nodes. This would make the system more flexible and manageable in a cloud environment.
3. **Comparison with Consensus Protocols:** For a more robust and formal guarantee against network partitions, implement a consensus protocol like Raft for leader election and replication(Elnozahy et al., 2002). This would move the system from a simple primary-backup model to a more robust replicated state machine.
4. **Production-Grade Implementation:** Port the system to a more performant language like Golang or C++ and integrate it with a real ML framework like PyTorch or TensorFlow to test its performance on actual large-scale training workloads.
5. **Handling Worker Failures:** Incorporate fault tolerance for worker nodes, perhaps using a task re-execution strategy similar to Hadoop, or a Byzantine-tolerant aggregation algorithm to handle potentially malicious or faulty worker updates

References

Computer, E. B.-, & 2012, undefined. (n.d.). CAP twelve years later: How the" rules" have changed. *Ieeexplore.Ieee.OrgE BrewerComputer, 2012•ieeexplore.Ieee.Org*. Retrieved July 12, 2026, from https://ieeexplore.ieee.org/abstract/document/6133253/?casa_token=ddC8eVpgnvEAAAAA:uInGoliFZb-JoX0vvDIcyeLC1LH6LSYS0vopGc5BqCwPlyUPmQWZuJlykd8Tu2ltXANYvw_uRA

Dean, J., Corrado, G. S., Monga, R., Chen, K., Devin, M., Le, Q. V, Mao, M. Z., Ranzato, A., Senior, A., Tucker, P., Yang, K., & Ng, A. Y. (n.d.). Large scale distributed deep networks. *Proceedings.Neurips.CcJ Dean, G Corrado, R Monga, K Chen, M Devin, M Mao, M Ranzato, A Senior, P TuckerAdvances in Neural Information Processing Systems, 2012•proceedings.Neurips.Cc*. Retrieved July 12, 2026, from https://proceedings.neurips.cc/paper_files/paper/2012/hash/6aca97005c68f1206823815f66102863-Abstract.html

DeCandia, G., Hastorun, D., Jampani, M., Kakulapati, G., Lakshman, A., Pilchin, A., Sivasubramanian, S., Vosshall, P., & Vogels, W. (2007). Dynamo: Amazon's highly available key-value store. *Dl.Acm.OrgG DeCandia, D Hastorun, M Jampani, G Kakulapati, A Lakshman, A PilchinACM SIGOPS Operating Systems Review, 2007•dl.Acm.Org*, *41*(6), 205–220. https://doi.org/10.1145/1323293.1294281

Elnozahy, E. N., Alvisi, L., Wang, Y. M., & Johnson, D. B. (2002). A survey of rollback-recovery protocols in message-passing systems. *Dl.Acm.OrgEN Elnozahy, L Alvisi, YM Wang, DB JohnsonACM Computing Surveys (CSUR), 2002•dl.Acm.Org*, *34*(3), 375–408. https://doi.org/10.1145/568522.568525

Network, M. V. S.-, & 2002, undefined. (n.d.). Distributed systems principles and paradigms. *People.Inf.Elte.Hu*. Retrieved July 12, 2026, from https://people.inf.elte.hu/toth_m/osztott_rendszerek_c/notes.01.pdf

Pang, B., Nijkamp, E., & Wu, Y. N. (2020). Deep learning with tensorflow: A review. *Journals.Sagepub.Com*, *45*(2), 227–248. https://doi.org/10.3102/1076998619872761

Appendices

A. Sample gRPC Protocol Definition (parameter_server.proto)

```
syntax = "proto3";

service ParameterService {
  rpc SendGradient(GradientRequest) returns (Ack);
  rpc GetParameters(Empty) returns (ParamReply);
  rpc FindMaster(Empty) returns (MasterInfo);
  rpc Heartbeat(Empty) returns (Ack);
}

message GradientRequest {
  repeated float gradients = 1;
  int32 worker_id = 2;
}

message ParamReply {
  repeated float parameters = 1;
}

message MasterInfo {
  string address = 1;
  int32 port = 2;
}

message Empty {}

message Ack {
  bool success = 1;
  string message = 2;
}
```

B. Snippet of Python (worker.py) Error Handling Logic

```
def run_training_loop(self):
    while self.iterations < 1000:
        try:
            # Get parameters
            params = self.stub.GetParameters(empty_pb2.Empty())
            # Compute synthetic gradient
            grad = self.compute_gradient(params.parameters)
            # Send gradient
            req = pb2.GradientRequest(gradients=grad, worker_id=self.id)
            self.stub.SendGradient(req)
            self.iterations += 1
        except grpc.RpcError as e:
            if e.code() == grpc.StatusCode.UNAVAILABLE:
                print(f"[Worker {self.id}] Primary unavailable. Discovering new master...")
                self.find_and_reconnect()
            else:
                print(f"An unexpected error occurred: {e}")
                time.sleep(1)

def find_and_reconnect(self):
    # Here, we would call the 'FindMaster' RPC to get the new primary's address.
    # For simplicity in this prototype, we assume a pre-defined static address.
    self.address = "localhost:50051"
    self.channel = grpc.insecure_channel(self.address)
    self.stub = pb2_grpc.ParameterServiceStub(self.channel)
    print(f"[Worker {self.id}] Reconnected to {self.address}")
```